

GenAI using LangChain - Deep Dive Notes (Code, Build Steps & Testing)

Deep-Dive Notes: How Code Works, Build Steps & Testing (Video by Video)

Compiled from CampusX playlist + detailed technical breakdown

- Introduction
- Video 1: GenAI Roadmap for Beginners
 - Kaise samjhein GenAI field ko (step-by-step)
 - “Testing” is stage pe
- Video 2: LangChain Components (Overview)
 - How LangChain works (concept level)
 - Steps to build your first LangChain setup
 - How to test
- Video 3: LangChain Models — Indepth Tutorial
 - How the code works
 - `temperature` kaise kaam karta hai (internally)
 - Steps to build (Embeddings demo)
 - How to test
- Video 4: Prompts in LangChain
 - How the code works
 - Steps to build a dynamic prompt system
 - How to test
- Video 5: Structured Output in LangChain
 - How the code works
 - Steps to build
 - How to test
- Video 6: Output Parsers in LangChain
 - How the code works
 - Steps to build
 - How to test
- Video 7: Chains in LangChain
 - How the code works
 - Steps to build a Sequential Chain (practical example)
 - How to test
- Video 8-9: Runnables in LangChain
 - How the code works
 - Steps to build a Branching Logic
 - How to test
- Video 10: Document Loaders in LangChain
 - How the code works
 - Steps to build a Multi-source Loader
 - How to test
- Video 11: Text Splitters in LangChain
 - How the code works
 - Steps to build

- How to test
- Video 12-13: Embeddings & Vector Stores
 - How the code works
 - Steps to build
 - How to test
- Video 14: Retrievers in LangChain
 - How the code works
 - Steps to build
 - How to test
- Video 15-16: Building a Complete RAG Pipeline
 - How the code works — full breakdown
 - Steps to build (poora RAG system from scratch)
 - How to test
- Video 17: Tool Calling in LangChain
 - How the code works
 - Steps to build a custom tool
 - How to test
- Video 18: Building End-to-End AI Agent
 - How the code works
 - Steps to build
 - How to test
- Video 19-21: Memory, Advanced Agents & Final Project
 - How Memory works (code + mechanics)
 - Steps to build a memory-enabled chatbot
 - How to test
 - Final Project — Testing Checklist (RAG + Tools + Memory combined)
- Overall Testing Philosophy (Summary)

Introduction

Ye notes pichhle summary se ek level aage hain. Har video ke liye ab teen cheezein cover hoti hain:

1. **How the code works** — line-by-line ya block-by-block explanation ki code actually andar kya kar raha hai
2. **Steps to build** — jaise ek beginner ko step-by-step guide diya jaaye ki wo khud se ye project/feature kaise banaye
3. **How to test** — us feature/code ko verify kaise karein ki sahi kaam kar raha hai (sample inputs, expected outputs, common errors)

Setup note (ek baar karna hai): Poore course mein ye base setup chahiye hoga —

```
pip install langchain langchain-openai langchain-community python-dotenv
```

`.env` file mein apni API key rakho: `OPENAI_API_KEY=sk-xxxx` — aur code mein `load_dotenv()` call karo taaki key load ho jaaye. Isse aage har video mein ye repeat nahi karenge.

Video 1: GenAI Roadmap for Beginners

Ye video **conceptual** hai (koi code nahi), isliye yahan “code work” nahi balki **mental-model build karne ke steps** hain.

Kaise samjhein GenAI field ko (step-by-step)

1. **Step 1 — Definition clear karo:** GenAI = naya content generate karne wali AI. Traditional ML se difference yaad rakho (predict vs create).
2. **Step 2 — Decide karo: User banna hai ya Builder?** Agar sirf tools use karne hain → prompting seekho. Agar apps banane hain → programming + frameworks seekho (ye course Builder path hai).
3. **Step 3 — Foundation vs Application split karo:** Pehle samjho LLM kaam kaise karta hai (thoda theory), phir seedha applications banana shuru karo (LangChain se) — dono ek saath mat karo, warna confusion hoga.
4. **Step 4 — Ek chhota goal set karo:** e.g., “Main ek PDF-chatbot banaunga” — is course ke end tak yahi target hai. Isse har video ka purpose clear rahega.

“Testing” is stage pe

Koi code nahi hai test karne ke liye — lekin apne aap ko test karo: - Kya tum explain kar sakte ho “GenAI aur traditional ML mein farak kya hai” kisi ko? - Kya tumhe pata hai tumhara goal (chatbot/agent/RAG app) kya hai is course ke baad?

Video 2: LangChain Components (Overview)

How LangChain works (concept level)

LangChain **components ko ek dusre se jodne (compose)** ka framework hai. Har component ek chhota kaam karta hai: - Model → sirf LLM se baat karta hai - Prompt → sirf template banata hai - Chain → in dono ko jodta hai

Jab tum inhe pipe (`|`) se jodte ho, LangChain ke andar ek internal object graph banta hai jisme: 1. Pehla component `.invoke(input)` call hota hai 2. Uska output **automatically** next component ka input ban jaata hai 3. Ye tab tak chalta hai jab tak chain khatam na ho

Isko samjho ek **pipeline of functions** ki tarah — jaise Unix mein `cat file | grep "error" | wc -l` — output ek se dusre mein pipe hota hai.

Steps to build your first LangChain setup

1. Install karo: `pip install langchain langchain-openai`
2. `.env` file banao aur API key daalo
3. Ek chhota Python file banao (`test.py`)
4. Model object banao aur `.invoke("Hello")` call karke check karo response aa raha hai ya nahi

```
from dotenv import load_dotenv
load_dotenv()
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o-mini")
response = model.invoke("Say hello in one word")
print(response.content)
```

How to test

- **Test 1 (Connectivity):** Upar wala code chalao — agar output print hota hai (error nahi aata), matlab API key aur installation sahi hai.
- **Test 2 (Error check):** Jaan-boojh kar galat model name daalo (e.g. `"gpt-abc"`) — dekho error message samajh aata hai ya nahi (ye debugging skill build karta hai).
- **Common Error:** `AuthenticationError` → API key galat/missing hai → `.env` file check karo aur `load_dotenv()` call hua hai ya nahi verify karo.

Video 3: LangChain Models — Indepth Tutorial

How the code works

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)
response = model.invoke("Explain gravity in one line")
print(response.content)
```

Line-by-line breakdown:

1. `ChatOpenAI(...)` — Ye ek **Python class object** banata hai jo andar hi andar OpenAI ke REST API endpoint (`/v1/chat/completions`) se connect hone ke liye configured hota hai. Is stage pe **koi API call nahi hoti** — sirf configuration set hoti hai (model name, temperature, API key from environment).
2. `model.invoke("Explain gravity in one line")` — Yahan actual **network call** hoti hai. Andar ye tumhare string ko ek message format mein convert karta hai: `json` `{"role": "user", "content": "Explain gravity in one line"}` aur ye JSON OpenAI ke server ko bhejta hai.
3. Server response wapas aata hai — ek `AIMessage` object ke form mein, jisme `.content` (actual text), `.response_metadata` (tokens used, model name, etc.) hote hain.
4. `response.content` — sirf text part print karta hai.

`temperature` kaise kaam karta hai (internally)

- LLM har next-word ke liye ek **probability distribution** predict karta hai (e.g., “The sky is ___” → “blue”: 70%, “clear”: 20%, “falling”: 1%...).
- `temperature=0` → hamesha sabse high-probability word choose hota hai (deterministic).
- `temperature=1` (ya zyada) → distribution “flatten” ho jaata hai, kam-probability words bhi chance paate hain → zyada creative/random output.

Steps to build (Embeddings demo)

1. `OpenAIEmbeddings()` object banao
2. `.embed_query("text")` call karo — ye ek text ko ek **fixed-length number list (vector)** mein convert karega (e.g., 1536 numbers)
3. Do alag texts ke vectors nikaal ke unka **cosine similarity** calculate karo — dekho similar-meaning texts ka similarity score zyada aata hai ya nahi

```
from langchain_openai import OpenAIEmbeddings
import numpy as np

embeddings = OpenAIEmbeddings()
v1 = embeddings.embed_query("I love programming")
v2 = embeddings.embed_query("I enjoy coding")
v3 = embeddings.embed_query("The weather is rainy today")

def cosine_similarity(a, b):
    a, b = np.array(a), np.array(b)
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print("Similar texts:", cosine_similarity(v1, v2)) # High score expected (~0.8+)
print("Unrelated texts:", cosine_similarity(v1, v3)) # Lower score expected
```

How to test

- **Test 1:** Run karke check karo `v1` aur `v2` ka similarity score `v1` aur `v3` se zyada hai — agar haan, embeddings sahi kaam kar rahe hain.
- **Test 2 (Length check):** `len(v1)` print karo — ye fixed number hona chahiye (model ke hisaab se, e.g. 1536) — har text ke liye same length aani chahiye.
- **Expected range:** Cosine similarity hamesha -1 se 1 ke beech hoti hai; similar texts ke liye typically 0.7-0.95, unrelated ke liye 0.1-0.5.

Video 4: Prompts in LangChain

How the code works

```
from langchain_core.prompts import PromptTemplate

template = PromptTemplate(
    template="Explain {topic} to a {audience} in simple words.",
    input_variables=["topic", "audience"]
)

prompt = template.invoke({"topic": "Blockchain", "audience": "5-year-old"})
print(prompt)
```

Kya ho raha hai andar: 1. `PromptTemplate(...)` — ek **template object** create hota hai jisme `{topic}` aur `{audience}` **placeholders** hain — Python ke f-string jaisa, lekin reusable object ke form mein. 2. `.invoke({"topic": "Blockchain", "audience": "5-year-old"})` — ye dictionary ke values ko placeholders mein **substitute** karta hai. Internally ye Python ke `str.format()` jaisa mechanism use karta hai, lekin ek `PromptValue` object return karta hai (raw string nahi) — taaki chain ke agle step (Model) isse directly consume kar sake. 3. Output: `"Explain Blockchain to a 5-year-old in simple words."`

Steps to build a dynamic prompt system

1. Decide karo tumhare prompt mein kaunse **variable parts** honge (jaise topic, tone, language, audience)
2. Un variables ko `{}` mein template string mein daalo
3. `input_variables` list mein sab variable names mention karo
4. Function banao jo user input leke `.invoke()` call kare
5. Model ke saath combine karo taaki poora pipeline test ho sake

```
def explain_topic(topic, audience):
    prompt = template.invoke({"topic": topic, "audience": audience})
    result = model.invoke(prompt)
    return result.content

print(explain_topic("Quantum Computing", "college student"))
```

How to test

- **Test 1 (Substitution check):** `template.invoke({...})` ka output print karo bina model ko call kiye — check karo variables sahi jagah replace hue ya nahi.
- **Test 2 (Missing variable error):** Jaan-boojh kar ek variable miss karo (sirf `topic` do, `audience` nahi) — LangChain error dega ("Missing input variable") — ye confirm karta hai validation kaam kar raha hai.
- **Test 3 (End-to-end):** Alag-alag `topic / audience` combinations ke saath function call karo aur manually check karo ki output audience ke hisaab se tone change ho raha hai (e.g., "5-year-old" ke liye simple language, "PhD student" ke liye technical language).

Video 5: Structured Output in LangChain

How the code works

```
from pydantic import BaseModel, Field

class MovieReview(BaseModel):
    title: str = Field(description="Movie name")
    rating: int = Field(description="Rating out of 10")
    summary: str

structured_model = model.with_structured_output(MovieReview)
result = structured_model.invoke("Review the movie Inception")
print(result.title, result.rating)
```

Andar kya ho raha hai (important — ye samajhna zaroori hai): 1. `class MovieReview(BaseModel)` — **Pydantic** ka use karke ek “schema” define kiya jaa raha hai — matlab ye batate ho ki output mein kaunse fields (`title` , `rating` , `summary`) honge aur unka data type kya hoga (str, int). 2. `model.with_structured_output(MovieReview)` — LangChain is schema ko convert karta hai ek **JSON Schema** mein aur usko LLM ko prompt ke saath bhejta hai — LLM ko bola jaata hai “sirf isi format mein reply karo.” 3. Modern LLMs (GPT-4, Gemini) ke paas ek special “**function calling**” / “**JSON mode**” capability hoti hai — model actual mein JSON text generate karta hai (as a string), jo LangChain ke andar automatically **parse aur validate** hota hai Pydantic model ke against. 4. Agar LLM ka output schema se match nahi karta (e.g., `rating` string bhej diya integer ki jagah), Pydantic **validation error** raise karega — isse aapko pata chal jaata hai kuch galat hai. 5. Final `result` ek Python object hai, string nahi — isliye `result.title` , `result.rating` directly access kar sakte ho (jaise normal Python class).

Steps to build

1. Decide karo tumhe output mein kaunse fields chahiye
2. Pydantic `BaseModel` class banao, har field ka type aur description do (description LLM ko guide karta hai ki field mein kya daalna hai)
3. `model.with_structured_output(YourSchema)` call karo
4. `.invoke()` karo aur object ke fields access karo

How to test

- **Test 1 (Type check):** `type(result)` print karo — `MovieReview` class ka instance aana chahiye, `str` nahi.
- **Test 2 (Field validation):** `result.rating` ka type check karo — `int` hona chahiye. Agar LLM ne “8/10” jaisa text generate kiya, Pydantic use fail kar dega ya coerce karega — ye dekhna important hai.
- **Test 3 (Edge case):** Ek aisi movie ka naam do jo exist nahi karti (e.g., “XYZ123”) — dekho model kya karta hai (hallucinate karega ya “not found” jaisa kuch boleگا) — ye samajhne mein help karta hai ki structured output bhi hallucination se immune nahi hai.

Video 6: Output Parsers in LangChain

How the code works

```
from langchain_core.output_parsers import StrOutputParser

parser = StrOutputParser()
chain = template | model | parser
result = chain.invoke({"topic": "AI", "audience": "kids"})
print(result) # result is now a plain string, not AIMessage object
```

Step-by-step internal flow: 1. `template | model | parser` — ye LCEL syntax hai. Python mein `|` operator ko LangChain ne **overload** kiya hai (`__or__` method define karke) taaki ye “connect these Runnables” ka matlab rakhe. 2. Jab `.invoke({"topic": "AI", "audience": "kids"})` call hota hai: - Pehle `template.invoke(...)` chalta hai → ek `PromptValue` return hota hai - Uska output automatically `model.invoke(PromptValue)` ko diya jaata hai → ek `AIMessage` object return hota hai (jisme `.content`, `.metadata` hota hai) - Uska output `parser.invoke(AIMessage)` ko diya jaata hai → `StrOutputParser` sirf `.content` nikal ke **plain string** return karta hai 3. Bina parser ke, `chain.invoke()` ka result ek `AIMessage` object hota — jisme se manually `.content` nikalna padta. Parser ye step automate kar deta hai.

JsonOutputParser kaise kaam karta hai: - Model ko instruction diya jaata hai (`parser.get_format_instructions()` se) ki JSON format mein reply de - Model text return karta hai jo JSON jaisa dikhta hai (e.g., `'{"name": "Raj", "age": 25}'`) - Parser is **string ko Python dictionary mein parse** karta hai `json.loads()` jaisa internal logic use karke - Agar model ne invalid JSON diya (e.g., trailing comma, missing quote), parser **error throw** karega

Steps to build

1. Decide karo kaunsa parser chahiye — plain text ke liye `StrOutputParser`, dictionary ke liye `JsonOutputParser`, validated object ke liye `PydanticOutputParser`
2. Agar `JsonOutputParser` / `PydanticOutputParser` use kar rahe ho, `parser.get_format_instructions()` ko apne prompt template mein add karo (taaki model ko pata ho kis format mein reply karna hai)
3. Chain banao: `prompt | model | parser`
4. Test karo alag-alag inputs ke saath

How to test

- **Test 1 (Basic run):** Chain ko run karo, output ka `type()` check karo — `StrOutputParser` ke liye `str`, `JsonOutputParser` ke liye `dict` aana chahiye.
- **Test 2 (Malformed output handling):** Kabhi-kabhi LLM invalid JSON generate karta hai — is case mein `try/except` laga ke dekho error properly catch ho raha hai ya nahi:

```
try:
    result = chain.invoke({"topic": "AI"})
except Exception as e:
    print("Parsing failed:", e)
```

- **Test 3 (Comparison):** Same input do — ek baar `StrOutputParser` ke saath, ek baar bina parser ke — dono outputs compare karo, difference samajh aayega (raw object vs clean string).

Video 7: Chains in LangChain

How the code works

```
chain = prompt | model | parser
result = chain.invoke({"topic": "AI", "audience": "kids"})
```

Sequential Chain internal working: - Har | ek naya `RunnableSequence` object banata hai jisme `steps` ek list mein store hote hain: `[prompt, model, parser]` - `.invoke(input)` call hone par, LangChain internally ek **loop** chalata hai: `python result = input for step in steps: result = step.invoke(result) return result` - Isse tumhe manually har step ka output next step ko pass nahi karna padta — LangChain automate kar deta hai.

Parallel Chain internal working:

```
from langchain_core.runnables import RunnableParallel

parallel_chain = RunnableParallel({
    "summary": summary_chain,
    "questions": question_chain
})
result = parallel_chain.invoke({"text": document})
```

- Andar, `RunnableParallel` **same input** (`{"text": document}`) ko `summary_chain` aur `question_chain` dono ko **ek saath** (concurrently — threading/async use karke) bhejta hai.
- Dono ka result ek dictionary mein collect hota hai: `{"summary": "...", "questions": "..."}"`
- Ye sequential se **fast** hota hai kyunki dono chains parallel mein execute hoti hain, ek ke baad ek nahi.

Steps to build a Sequential Chain (practical example)

1. Ek prompt banao jo essay generate kare
2. Ek dusra prompt banao jo essay ko summarize kare
3. Dono ko chain karo taaki ek ka output dusre ka input bane

```
essay_prompt = PromptTemplate(template="Write a short essay on {topic}", input_variables=["topic"])
summary_prompt = PromptTemplate(template="Summarize this in 2 lines:\n{essay}", input_variables=["essay"])

essay_chain = essay_prompt | model | parser
full_chain = essay_chain | (lambda essay: {"essay": essay}) | summary_prompt | model | parser

result = full_chain.invoke({"topic": "Climate Change"})
```

Note: Yahan ek `lambda` use kiya gaya hai output ko reshape karne ke liye — kyunki `summary_prompt` ko `{"essay": ...}` dict chahiye, lekin `essay_chain` sirf ek string return karta hai. Ye ek common real-world pattern hai.

How to test

- **Test 1 (Intermediate output check):** Har step ka output alag se print karo (`essay_chain.invoke(...)` akela run karke dekho) — taaki pata chale error kahan aa raha hai agar poori chain fail ho.

- **Test 2 (Parallel speed test):** Sequential aur Parallel chain dono ka execution time compare karo (`time.time()` use karke) — Parallel wala tez hona chahiye.
- **Test 3 (Different topics):** 3-4 alag topics de kar chain chalao, dekho output format consistent rehta hai ya nahi.

Video 8-9: Runnables in LangChain

How the code works

```
from langchain_core.runnables import RunnableLambda, RunnablePassthrough, RunnableBranch

double = RunnableLambda(lambda x: x * 2)
print(double.invoke(5)) # 10
```

Kya ho raha hai: `RunnableLambda` ek normal Python function ko ek “Runnable” object mein **wrap** kar deta hai — taaki wo bhi `.invoke()`, `.batch()`, aur `chain()` syntax support kare, jaise koi built-in LangChain component karta hai. Ye custom logic ko pipeline mein fit karne ka tarika hai.

```
from langchain_core.runnables import RunnablePassthrough

chain = RunnableParallel({
    "original": RunnablePassthrough(),
    "doubled": RunnableLambda(lambda x: x["number"] * 2)
})
result = chain.invoke({"number": 5})
# {"original": {"number": 5}, "doubled": 10}
```

`RunnablePassthrough` — input ko **as-it-is** wapas return kar deta hai, kuch process nahi karta. Ye tab useful hota hai jab chain ke ek branch mein original input ko unchanged aage bhejna ho (RAG mein dekhoge — user ka original question bina change kiye prompt tak pahunchana).

```
from langchain_core.runnables import RunnableBranch

branch = RunnableBranch(
    (lambda x: x["length"] > 500, long_summary_chain),
    (lambda x: x["length"] <= 500, short_summary_chain),
    default_chain
)
```

`RunnableBranch` internally har condition (lambda function) ko **order mein check** karta hai — jo pehla condition `True` return kare, uska corresponding chain execute hota hai (if-elif-else jaisa). Agar koi condition match na ho, `default_chain` run hota hai.

Steps to build a Branching Logic

1. Decide karo condition kya hogi (e.g., text length, sentiment, category)
2. Har condition ke liye ek alag chain define karo
3. `RunnableBranch` mein tuples ke form mein `(condition_function, chain)` pass karo
4. Last argument mein default chain do (fallback)

How to test

- **Test 1 (Each branch individually):** Manually har condition ko trigger karne wale inputs do (e.g., ek chhota text, ek bada text) — verify karo sahi chain trigger hui.
- **Test 2 (Default fallback):** Ek aisa input do jo koi bhi condition match na kare — dekho `default_chain` chalta hai ya nahi.
- **Test 3 (RunnableLambda type-safety):** Function ke andar galat key access karo (jaise `x["missing_key"]`) — `KeyError` aana chahiye — ye batata hai error-handling add karne ki zaroorat hai (try/except).

Video 10: Document Loaders in LangChain

How the code works

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("resume.pdf")
docs = loader.load()
print(len(docs))          # Number of pages
print(docs[0].page_content) # Text of first page
print(docs[0].metadata)    # {'source': 'resume.pdf', 'page': 0}
```

Andar kya hota hai: 1. `PyPDFLoader("resume.pdf")` — file path store karta hai, file ko turant nahi padhta. 2. `.load()` call hone par — ye andar `pypdf` library use karke PDF ko page-by-page open karta hai, har page se text **extract** karta hai (text-layer se, images se nahi). 3. Har page ek **Document object** ban jaata hai jisme do cheezein hoti hain: `page_content` (extracted text) aur `metadata` (source file, page number — jo baad mein useful hota hai citation dikhane ke liye). 4. `docs` ek list hai — list ki length = PDF mein pages ki sankhya.

Steps to build a Multi-source Loader

1. Decide karo kaunse sources se data lena hai (PDF, website, CSV)
2. Har source ke liye appropriate loader import karo
3. `.load()` call karke `Document` objects ka list nikaalo
4. Sabko ek single list mein combine karo (agar multiple sources hain)

```
from langchain_community.document_loaders import PyPDFLoader, WebBaseLoader

pdf_docs = PyPDFLoader("company_policy.pdf").load()
web_docs = WebBaseLoader("https://example.com/faq").load()

all_docs = pdf_docs + web_docs
print(f"Total documents loaded: {len(all_docs)}")
```

How to test

- **Test 1 (Content sanity check):** `docs[0].page_content[:200]` print karo — check karo readable text aa raha hai, garbage characters nahi (agar scanned PDF hai toh text empty ya garbled aayega — iska matlab OCR chahiye hoga).
- **Test 2 (Metadata check):** `docs[0].metadata` print karo — `source` aur `page` fields honi chahiye, taaki baad mein RAG mein “answer PDF ke page 3 se aaya” jaisa citation de sako.
- **Test 3 (Empty file handling):** Ek empty ya corrupt PDF de kar dekho error handle ho raha hai (`try/except` laga ke).

Video 11: Text Splitters in LangChain

How the code works

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)
print(f"Original docs: {len(docs)}, After splitting: {len(chunks)}")
```

Andar RecursiveCharacterTextSplitter kaise kaam karta hai (important detail): 1. Ye ek list of separators try karta hai **order mein**: pehle `"\n\n"` (paragraph break), fir `"\n"` (line break), fir `" "` (space), fir individual characters. 2. Sabse pehle poore text ko `"\n\n"` se split karne ki koshish karta hai. 3. Agar resulting chunk `chunk_size` (500 characters) se bada hai, toh us chunk ko agle separator (`"\n"`) se further split karta hai. 4. Ye recursively chalta hai jab tak har chunk `chunk_size` ke andar fit na ho jaaye. 5. `chunk_overlap=50` — har naye chunk ke start mein pichhle chunk ke last 50 characters repeat hote hain, taaki agar koi important sentence chunk-boundary pe kat raha ho, toh context dono chunks mein preserve rahe.

Isliye "Recursive" naam hai: kyunki ye baar-baar (recursively) chhote separators try karta hai jab tak sahi size na mil jaaye — bina beech mein words/sentences ko randomly tode.

Steps to build

1. Apna `chunk_size` decide karo — chhota size (200-500) precise retrieval ke liye better, bada size (1000+) zyada context deta hai but less precise.
2. `chunk_overlap` set karo — typically `chunk_size` ka 10-20%.
3. `splitter.split_documents(docs)` call karo.
4. Chunks ka size distribution check karo.

How to test

- **Test 1 (Chunk count):** `len(chunks)` check karo — original docs se zyada hona chahiye (kyunki har document multiple chunks mein tuta hai).
- **Test 2 (Size verification):** Loop chala ke check karo koi chunk `chunk_size` se bahut zyada bada toh nahi (kuch overshoot normal hai agar koi single word/sentence bahut lamba ho):

```
for c in chunks:
    if len(c.page_content) > 600: # chunk_size + buffer
        print("Oversized chunk found!", len(c.page_content))
```

- **Test 3 (Overlap check):** Do consecutive chunks ka text print karke manually verify karo ki end/start mein overlap ho raha hai ya nahi.
- **Test 4 (Sentence-integrity check):** Kuch chunks manually padho — check karo koi sentence beech mein bilkul random jagah pe toh nahi kat raha (agar kat raha hai, `chunk_size` badhao).

Video 12-13: Embeddings & Vector Stores

How the code works

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma

embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(chunks, embeddings)
```

Step-by-step internal process (ye samajhna sabse important hai poore RAG mein): 1.

`Chroma.from_documents(chunks, embeddings)` — internally ye har `chunk` (jo Video 11 mein bane the) ko loop karta hai. 2. Har chunk ke `page_content` (text) ko `embeddings.embed_documents()` ko bhejta hai — ye OpenAI API ko call karta hai aur har chunk ka ek **vector** (numbers ki list, e.g., 1536 dimensions) wapas laata hai. 3. Ye vector + original text + metadata sab **Chroma database** (jo disk pe ya memory mein store hoti hai) mein save ho jaate hain — usually ek special data-structure (like HNSW index) mein, jo fast similarity-search ke liye optimized hoti hai. 4. `vectorstore` object ab ek “searchable index” ban chuka hai.

Search kaise hoti hai (`similarity_search`):

```
results = vectorstore.similarity_search("What is the refund policy?", k=3)
```

1. Query text ("What is the refund policy?") ka bhi embedding banta hai (same embedding model se).
2. Vector store apne stored har chunk-vector se query-vector ki **cosine similarity / distance** calculate karta hai.
3. Sabse zyada similar (closest) `k=3` chunks return kiye jaate hain — sorted by similarity score.

Kyun important hai same embedding model use karna? Kyunki different models ka vector-space alag hota hai — agar chunks ek model se embed hue aur query dusre model se, distances meaningless ho jaayengi.

Steps to build

1. Chunks ready karo (Video 10-11 se)
2. Embedding model choose karo (`OpenAIEmbeddings` , `HuggingFaceEmbeddings` , etc.)
3. Vector store banao (`Chroma` , `FAISS`) aur chunks ko `.from_documents()` se load karo
4. Optionally, disk pe persist karo taaki dobara embed na karna pade:

```
vectorstore = Chroma.from_documents(chunks, embeddings, persist_directory="./db")
```

How to test

- **Test 1 (Basic retrieval sanity):** Ek query do jiska answer chunks mein clearly present hai — check karo returned chunks mein wo relevant text hai:

```
results = vectorstore.similarity_search("refund policy", k=2)
for r in results:
    print(r.page_content[:150], "\n---")
```


- **Test 2 (Irrelevant query test):** Ek bilkul unrelated query do (jo document se related hi nahi) — dekho vector store kya return karta hai (kabhi-kabhi “best available match” return karega, chahe wo actually relevant na ho — ye ek known limitation hai jo aage retrievers/threshold se handle hoti hai).
- **Test 3 (Score check):** `similarity_search_with_score()` use karke actual similarity numbers dekho — relevant results ka score threshold decide karne mein help karta hai.
- **Test 4 (Persistence test):** Vector store ko save karke, naya Python session start karo, `Chroma(persist_directory="./db", embedding_function=embeddings)` se reload karo — check karo data preserved hai.

Video 14: Retrievers in LangChain

How the code works

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
relevant_docs = retriever.invoke("What is the refund policy?")
```

Andar kya ho raha hai: 1. `vectorstore.as_retriever(...)` — vector store ko ek **Runnable Retriever object** mein wrap karta hai. Isse fayda ye hai ki ab ye bhi chain (`|`) syntax mein directly use ho sakta hai, jaise model ya parser. 2. `.invoke(query)` call hone par, andar wahi `similarity_search()` chalta hai jo Video 12-13 mein dekha, aur top- k `Document` objects return karta hai.

MMR (search_type="mmr") kaise kaam karta hai:

```
retriever = vectorstore.as_retriever(search_type="mmr", search_kwargs={"k": 3, "fetch_k": 10})
```

1. Pehle zyada candidates fetch kiye jaate hain (`fetch_k=10`) — top 10 most-similar chunks.
2. Fir ek **greedy algorithm** chalta hai: sabse pehla result sabse relevant chunk hota hai. Uske baad, agla result choose karte time algorithm dekhta hai — “ye chunk relevant bhi ho AND pehle-selected chunks se zyada **alag (diverse)** bhi ho.”
3. Ye process `k` results milne tak chalta hai — isse output mein redundancy kam hoti hai.

Steps to build

1. Vector store ready karo
2. `.as_retriever()` call karo, `k` (kitne results chahiye) aur `search_type` (`"similarity"` ya `"mmr"`) decide karo
3. Retriever ko test karo standalone (bina chain ke) pehle

How to test

- **Test 1 (k-value check):** `len(relevant_docs)` check karo — `k` ke barabar hona chahiye (jab tak documents kam na ho).
- **Test 2 (Similarity vs MMR comparison):** Same query dono search_types ke saath run karo, results compare karo — MMR wale results mein zyada topic-diversity honi chahiye.
- **Test 3 (Chain integration test):** Retriever ko chain mein daal ke check karo ye seamlessly kaam karta hai:

```
chain = retriever | (lambda docs: "\n".join([d.page_content for d in docs]))
print(chain.invoke("refund policy"))
```

Video 15-16: Building a Complete RAG Pipeline

How the code works — full breakdown

```
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

prompt = PromptTemplate(
    template="Answer using only this context:\n{context}\n\nQuestion: {question}",
    input_variables=["context", "question"]
)

rag_chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | model
    | StrOutputParser()
)

answer = rag_chain.invoke("What is the leave policy of the company?")
```

Step-by-step execution flow (bahut important hai isko samajhna): 1. `rag_chain.invoke("What is the leave policy of the company?")` — ye string input hai. 2. Sabse pehle `{"context": retriever, "question": RunnablePassthrough()}` — ye ek **dictionary of Runnables** hai. LangChain **dono ko parallel mein** run karta hai, **same input** (poora question string) dono ko deta hai: - `retriever.invoke("What is the leave policy...")` → relevant `Document` objects ka list return karta hai - `RunnablePassthrough().invoke("What is the leave policy...")` → input ko as-it-is wapas deta hai (yahan poora question string) - Result: `{"context": [Document(...), Document(...)], "question": "What is the leave policy..."}` 3. **Problem:** `context` abhi `Document` objects ki list hai, plain text nahi — prompt ko plain text chahiye. (Real production code mein isko fix karne ke liye ek formatting step add karte hain — video mein ye dikhaya jaata hai):

```
def format_docs(docs):
    return "\n\n".join(doc.page_content for doc in docs)

rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | model
    | StrOutputParser()
)
```

Yahan `retriever | format_docs` — retriever ka output (Document list) seedha `format_docs` function (jo `RunnableLambda` ki tarah act karta hai) ko chala jaata hai, jo unhe ek single string mein combine kar deta hai. 4. Ye dictionary (`{"context": "...", "question": "..."}`) ab `prompt.invoke(...)` ko jaata hai → placeholders fill hote hain → final formatted prompt banta hai. 5. Ye prompt `model.invoke(...)` ko jaata hai → LLM us context ko padh ke, us context ke basis pe answer generate karta hai (na ki apni training memory se). 6. `StrOutputParser()` output ko clean string mein convert karta hai.

Steps to build (poora RAG system from scratch)

1. Documents load karo (Video 10)

2. Chunks banao (Video 11)
3. Embeddings + Vector store banao (Video 12-13)
4. Retriever banao (Video 14)
5. Prompt template banao jisme explicitly likha ho “sirf diye gaye context se answer do” (isse hallucination kam hoti hai)
6. `format_docs` function banao (documents ko text mein convert karne ke liye)
7. Poora chain jodo: `{context, question} | prompt | model | parser`
8. Conversational RAG ke liye — chat history bhi dictionary mein add karo aur prompt mein include karo

How to test

- **Test 1 (Grounding test):** Ek aisa question pucho jiska answer document mein clearly likha hai — verify karo answer document ke content se match karta hai (word-for-word nahi, but factually).
- **Test 2 (Hallucination test — important!):** Ek aisa question pucho jiska answer document mein **hai hi nahi** — check karo model “I don’t know” / “not mentioned in the document” jaisa bolta hai ya galat se kuch bana ke bol deta hai (agar bana ke bol raha hai, prompt ko aur strict banao: “If the answer is not in the context, say ‘I don’t know’”).
- **Test 3 (Context inspection):** `format_docs` se pehle print karke dekho retriever ne kaunse chunks return kiye — agar wrong chunks aa rahe hain, retrieval step ki problem hai (`chunk_size`, `k`, ya embedding model change karna pad sakta hai).
- **Test 4 (End-to-end multiple queries):** 5-10 different questions ka ek test-set banao, sabko chain se run karo, manually accuracy check karo — isko “manual evaluation” kehte hain, real projects mein zaroori step hai.

Video 17: Tool Calling in LangChain

How the code works

```
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """Get current weather of a city"""
    return f"The weather in {city} is 30°C and sunny."

llm_with_tools = model.bind_tools([get_weather])
response = llm_with_tools.invoke("What's the weather in Delhi?")
print(response.tool_calls)
```

Line-by-line andar ki mechanics:

1. `@tool` decorator — ye function ko ek **Tool object** mein convert karta hai. Ye function ki **docstring** (`"""Get current weather..."""`), **naam** (`get_weather`), aur **parameter types** (`city: str`) — sabko automatically ek schema mein convert kar deta hai jo LLM samajh sake. **Isliye docstring likhna zaroori hai** — LLM isi se samajhta hai tool kab use karna hai.
2. `model.bind_tools([get_weather])` — Ye tool ka schema LLM ko available tools ki list ke saath “attach” kar deta hai. Har request ke saath ye schema bhi model ko bheja jaata hai.
3. `.invoke("What's the weather in Delhi?")` — LLM query padhta hai, decide karta hai “iske liye mujhe `get_weather` tool chahiye, aur argument `city="Delhi"` hoga.” **Lekin LLM khud function call NAHI karta** — wo sirf ek structured response deta hai: `json { "name": "get_weather", "args": { "city": "Delhi" } }`
4. `response.tool_calls` — ye wahi structured decision hota hai, Python list/dict ke form mein.
5. **Actual execution** developer ko manually karni padti hai:

```
for call in response.tool_calls:
    if call["name"] == "get_weather":
        result = get_weather.invoke(call["args"])
        print(result)
```

Steps to build a custom tool

1. Ek normal Python function likho jo koi kaam kare (API call, calculation, database query)
2. Uspar `@tool` decorator lagao
3. **Clear docstring likho** — ye batati hai LLM ko tool kab/kaise use karna hai (ye sabse common mistake area hai beginners ke liye)
4. Type hints zaroor do (`city: str`) — LLM inhi se samajhta hai kaunsa argument bhejna hai
5. `model.bind_tools([tool1, tool2, ...])` se sab tools bind karo
6. Response ke `.tool_calls` ko manually execute karne ka logic likho

How to test

- **Test 1 (Tool selection check):** Alag-alag queries do jinme se sirf kuch weather-related hon — check karo model sahi query pe hi tool select karta hai:

```
r1 = llm_with_tools.invoke("What's the weather in Mumbai?")
print(r1.tool_calls) # Should call get_weather

r2 = llm_with_tools.invoke("What is 2+2?")
print(r2.tool_calls) # Should be empty (no tool needed)
```

- **Test 2 (Argument extraction check):** Verify karo `call["args"]["city"]` sahi city name

extract kar raha hai (typos, alternate spellings ke saath test karo).

- **Test 3 (Manual execution check):** Tool ko manually call karke (bina LLM ke) verify karo function khud sahi kaam karta hai — isolated unit test jaisa.
- **Test 4 (No-tool-needed case):** Ek aisa question pucho jisko koi tool ki zaroorat hi nahi — check karo `response.tool_calls` empty aata hai aur model normal text answer deta hai.

Video 18: Building End-to-End AI Agent

How the code works

```
from langchain.agents import create_tool_calling_agent, AgentExecutor

agent = create_tool_calling_agent(model, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

result = agent_executor.invoke({"input": "What's the weather in Delhi and what is 15% of that temperature?"})
print(result["output"])
```

Andar poora ReAct loop kaise chalta hai (step-by-step): 1. `agent_executor.invoke(...)` call hote hi, LLM ko query di jaati hai + saare available tools ki list. 2. **Iteration 1:** LLM sochta hai (reasoning) — “Pehle mujhe weather chahiye” → `get_weather` tool call karne ka decision leta hai. 3. `AgentExecutor` is decision ko **automatically detect** karta hai (jo Video 17 mein tumhe manually karna pada tha, ye class wo kaam khud karti hai), tool ko **actually execute** karta hai, result leta hai (e.g., “30°C”). 4. Ye result LLM ko wapas bheja jaata hai — as a “tool result” message, conversation history mein add hoke. 5. **Iteration 2:** LLM ab sochta hai — “Ab mujhe 30 ka 15% calculate karna hai” → agar calculator tool available hai toh use call karta hai, warna khud math kar leta hai. 6. Ye loop tab tak chalta hai (`verbose=True` se tum ye poora reasoning + action + observation cycle console mein dekh sakte ho) jab tak LLM decide na kare “ab mere paas final answer hai.” 7. `result["output"]` — final natural-language answer hota hai.

`verbose=True` **ka use:** Development ke time ye ON rakhna chahiye — isse tumhe console mein poora “thought process” dikhta hai (Thought → Action → Observation → Thought → ... → Final Answer) — debugging ke liye bahut helpful.

Steps to build

1. Apne tools define karo (Video 17 pattern follow karke, multiple tools ek saath)
2. Ek agent-specific prompt template banao (jisme system instructions ho ki agent kaise behave kare)
3. `create_tool_calling_agent(model, tools, prompt)` se agent banao
4. `AgentExecutor` mein wrap karo (ye actual execution/looping handle karta hai)
5. `.invoke({"input": "..."})` se test karo

How to test

- **Test 1 (Single-tool query):** Simple query do jisme sirf ek tool chahiye — verify karo agent sahi tool use karke sahi answer deta hai.
- **Test 2 (Multi-tool/multi-step query):** Complex query do jisme 2+ tools sequentially chahiye (jaise upar wala weather+percentage example) — `verbose=True` se dekho agent sahi steps mein tools chain kar raha hai ya nahi.
- **Test 3 (No-tool query):** Ek general knowledge question do jisme koi tool ki zaroorat nahi — verify karo agent unnecessarily tool call nahi karta (efficiency check).
- **Test 4 (Infinite loop check):** Kabhi-kabhi agent confuse ho ke baar-baar same tool call karta rehta hai — `max_iterations` parameter set karo (`AgentExecutor(..., max_iterations=5)`) aur test karo ye limit properly enforce hoti hai.
- **Test 5 (Error resilience):** Ek tool mein jaan-boojh kar bug daalo (e.g., exception raise kare)

— dekho agent us error ko gracefully handle karta hai ya crash ho jaata hai — production mein `handle_parsing_errors=True` jaise options useful hote hain.

Video 19-21: Memory, Advanced Agents & Final Project

How Memory works (code + mechanics)

```
chat_history = []

def chat(user_input):
    chat_history.append({"role": "user", "content": user_input})
    response = model.invoke(chat_history)
    chat_history.append({"role": "assistant", "content": response.content})
    return response.content

print(chat("My name is Raj"))
print(chat("What is my name?"))  # Model should correctly say "Raj"
```

Andar kya ho raha hai: 1. LLM **fundamentally stateless** होता है — har `.invoke()` call एक **naya, independent** request होती है। Model को “yaad” कुछ नहीं रहता अपने आप। 2. “Memory” का पूरा concept ये है कि **har naye call mein poori purani conversation history bhi bhej do** — model को लगता है जैसे उसे “yaad” है, लेकिन actually तुम हर बार पूरा context reprovide कर रहे हो। 3. `chat_history` list में हर message (`role: user/assistant`) sequentially store होता है, और `model.invoke(chat_history)` पूरी list एक साथ भेजता है — LLM last message का jawab deta है, लेकिन **poori history ko context ki tarah use karke**.

⚠ **Practical Issue:** जैसे-जैसे conversation लंबी होती है, `chat_history` भी बड़ी होती जाती है — और LLMs की एक **token limit** होती है। Isliye production systems में history को **summarize** या **truncate** (last N messages ही rakhna) करना पड़ता है।

Steps to build a memory-enabled chatbot

1. Ek list/session-object बनाओ जो conversation store करे
2. Har user message को history में add करो, फिर पूरी history model को भेजो
3. Model का response भी history में add करो (taaki agla turn use dekh sake)
4. Agar history बहुत लंबी हो जाये, purane messages summarize/trim करो

How to test

- **Test 1 (Basic recall test):** “My name is X” बोलो, फिर अगले message में “What is my name?” पूछो — verify करो model correct नाम बताता है।
- **Test 2 (Without memory comparison):** Same 2 messages को **bina history maintain kiye** (हर बार fresh list) भेजो — देखो model “I don’t know your name” बताता है — ये clearly दिखता है memory का fark.
- **Test 3 (Token overflow test):** Ek बहुत लंबी conversation simulate करो (50+ messages) — देखो कहां पे error आता है (token limit exceed) — इसे पता चलेगा truncation/summarization kab implement करनी है।

Final Project — Testing Checklist (RAG + Tools + Memory combined)

Jab teeno combine karke final chatbot बनाओ, ये systematic testing checklist follow करो: 1. ✓

Document-grounded questions — document से related sawaal sahi answer aate hain 2. ✓ **Out-**

of-scope questions — document se bahar ke sawaal pe model “I don’t know” bolta hai, hallucinate nahi karta 3. ✓ **Tool-triggering questions** — jab calculation/live-data chahiye, agent sahi tool use karta hai 4. ✓ **Follow-up/contextual questions** — pichhle message ka reference samajh ke sahi jawab deta hai (memory working) 5. ✓ **Edge cases** — empty input, bahut lamba input, special characters — sab gracefully handle hote hain 6. ✓ **Latency check** — response time acceptable hai (2-5 seconds range typically)

Overall Testing Philosophy (Summary)

Har LangChain component test karte waqt ye general pattern follow karo:

Layer	Test Approach
Model	Simple <code>.invoke()</code> se connectivity check
Prompt	Substitution manually verify karo (bina model call kiye)
Parser	Output ka <code>type()</code> check karo, malformed-input se error-handling test karo
Chain	Har intermediate step ko individually test karo, phir end-to-end
Retriever/Vector Store	Known-answer queries se relevance verify karo
RAG	Grounding test + Hallucination test (out-of-scope query)
Tools	Manual isolated test + LLM tool-selection test alag-alag
Agent	Single-tool, multi-tool, no-tool, aur error-case — chaaron scenario test karo
Memory	With vs without history comparison karke fark verify karo

 **Golden Rule:** Kabhi bhi poori chain seedhe test mat karo agar wo fail ho rahi ho — **step-by-step break karke** har component ko individually `.invoke()` karke test karo, taaki pata chale problem kahan hai (prompt mein, model mein, ya parsing mein).

Notes compiled from CampusX’s “Generative AI using LangChain” playlist, with detailed code mechanics, build steps, and testing methodology added for deeper, practical understanding.